

Geospatial Flood-Risk Modelling using Climate Data and Hidden Markov Models in Rwanda

Nyabugogo–Nyabarongo corridor, Kigali

Progress presentation · ML gap → solution → status → prototype

Kellen Murerwa | **Supervisor: Emmanuel Adjei**

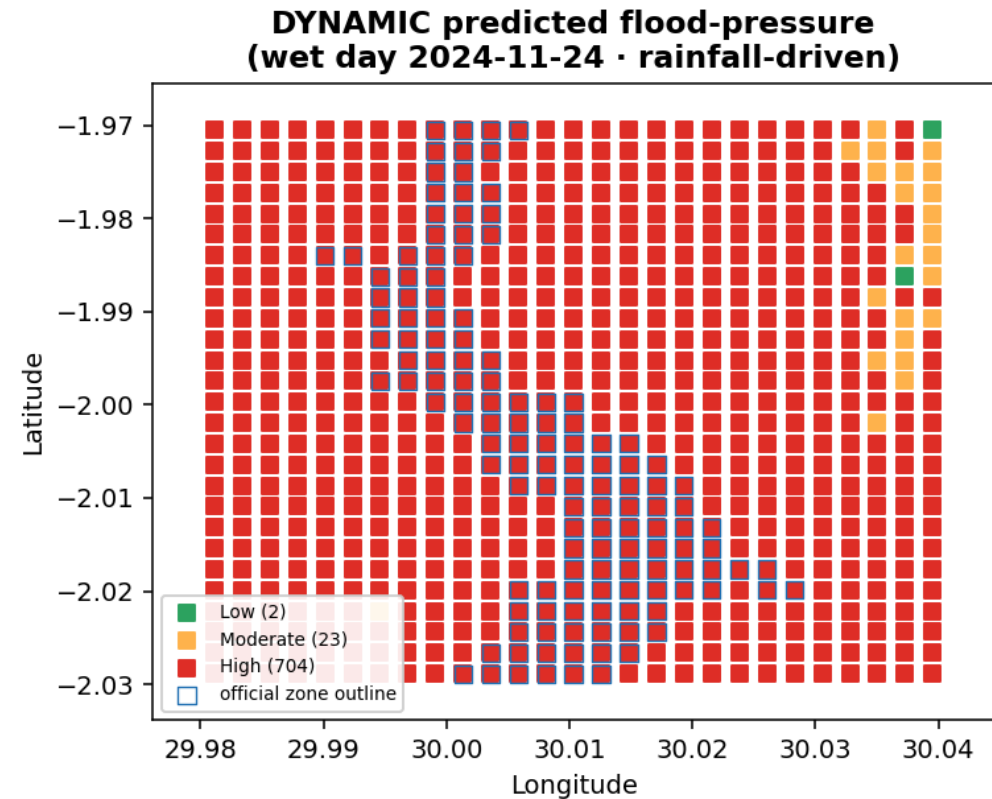
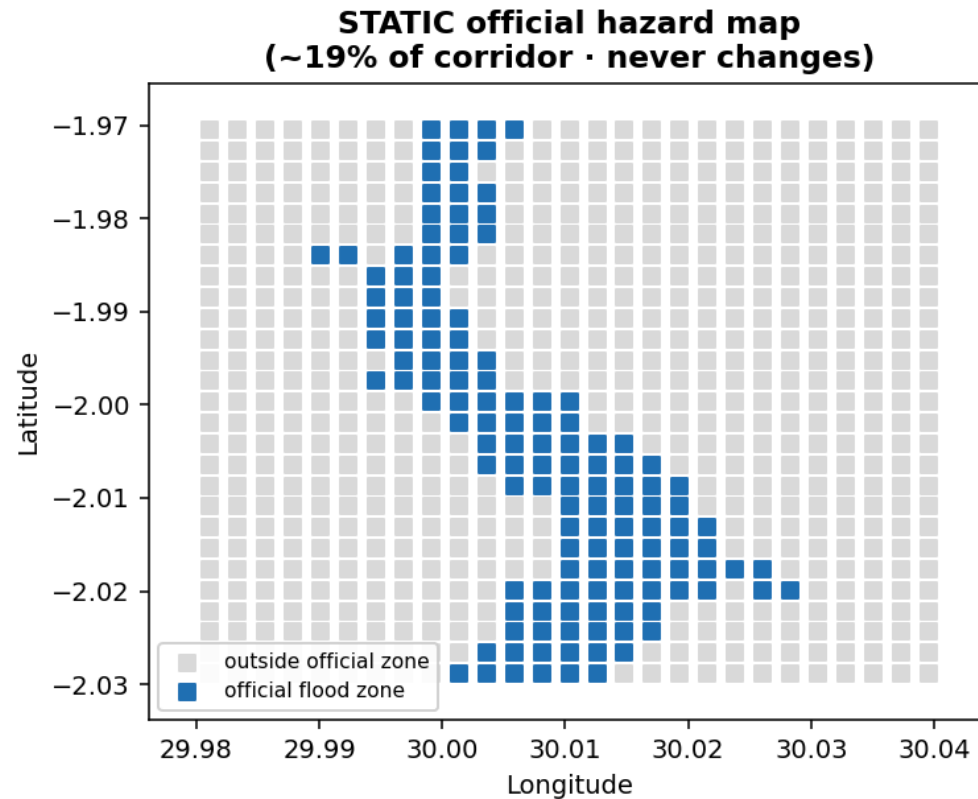
BSc Software Engineering · African Leadership University

The ML gap we address

- Kigali's existing flood-risk tools are STATIC hazard maps — they cover only ~19% of the study corridor and never change with the weather.
 - They give no time-aware, day-to-day view of where flood pressure is building after heavy rain.
- ML flood-susceptibility studies exist, but in this setting they rarely:
 - combine supervised geospatial ML with a temporal (sequence) model,
 - rely only on open / free data for a data-scarce African urban catchment,
 - or ship a validated, usable artefact — most stop at an accuracy table.
- → The gap: a reproducible, time-aware ML system from OPEN data that beats static maps and rule baselines — and is validated against the official flood polygons.

Static map vs. dynamic prediction

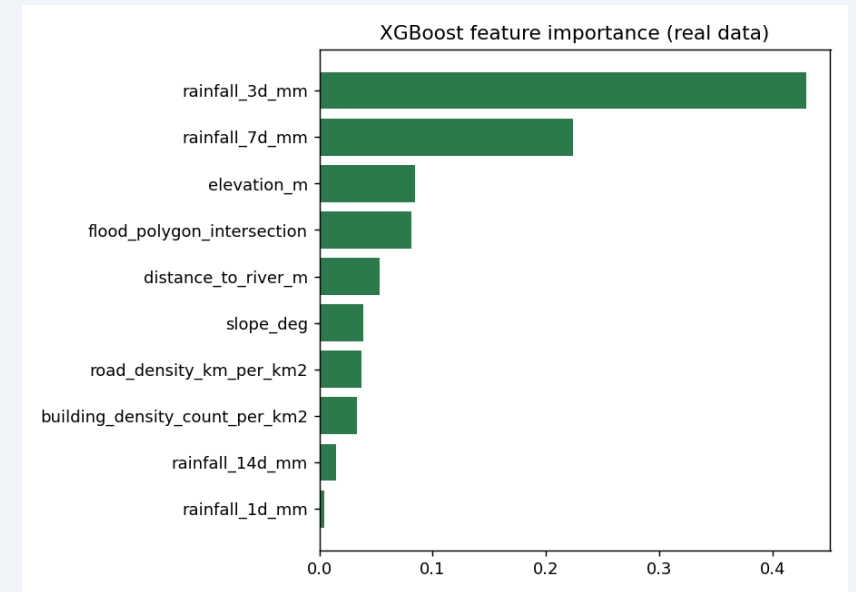
The gap: a fixed map vs. a time-aware, data-driven risk view



Left: the fixed official zone (same every day). Right: on a wet day the model flags a far broader, rainfall-driven footprint — the risk the static map misses.

Proposed solution

- A 250 m grid over the corridor (729 cells), one row per cell-per-day, 2018–2024 (1.86 M cell-days).
- Fuse four open, keyless sources into 10 features:
 - rainfall accumulations (1/3/7/14-day) — Open-Meteo ERA5,
 - terrain (elevation, slope) — SRTM; exposure (river, roads, buildings) — OSM,
 - official flood-zone membership — Rwanda GeoPortal / MOE.
- Predict daily Low / Moderate / High flood-pressure states.
- Benchmark LR/DT, Random Forest, XGBoost vs. rainfall-threshold & static-polygon baselines; add an HMM temporal layer.
- Honest test: temporal split — train 2018–22, validate 2023, test on unseen 2024; then spatial validation + a Streamlit dashboard.



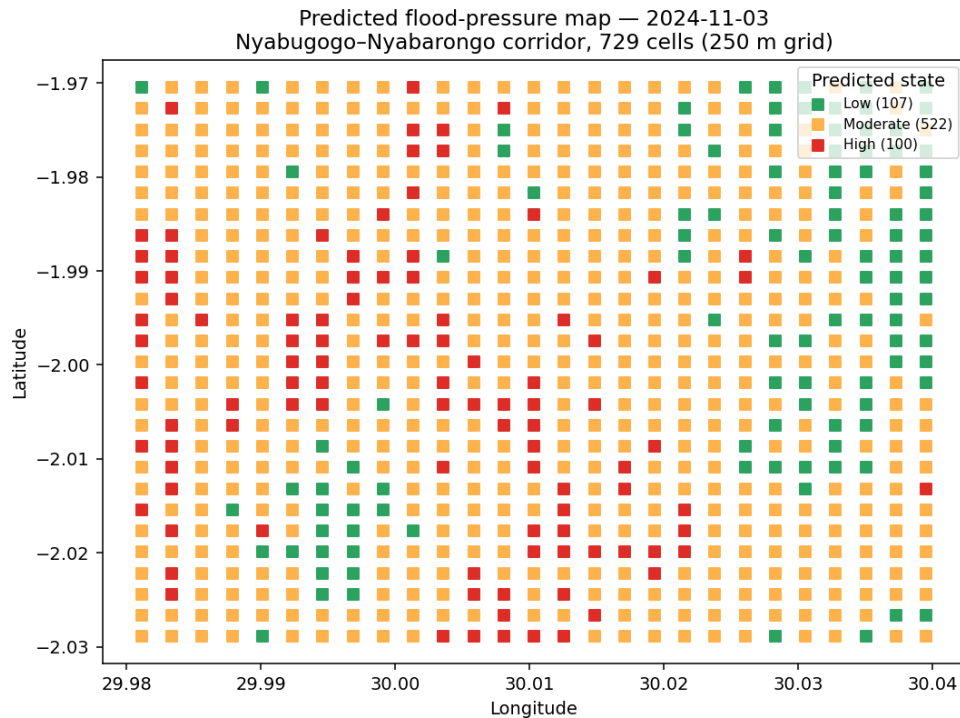
Engineered features ranked by importance

Current status

- Data pipeline COMPLETE — 1.86 M real cell-days, fully reproducible (cached, offline-friendly).
- Models trained & benchmarked on the 2024 test hold-out — targets MET.
- XGBoost (deployed) macro-F1 0.813 / High-recall 0.843; Random Forest 0.813 / 0.849.
- Train $\approx$ val $\approx$ test macro-F1 — no over-fitting. Beats baselines: +0.19 F1 over rainfall, +0.49 over static map.
- Spatial validation DONE — enrichment 1.60 $\times$ (≥ 1.4 ) , inside/outside odds 1.85 $\times$ (≥ 1.5 ) .
- HMM temporal layer integrated (descriptive; supervised layer is the predictor).
- Working dashboard + 10/10 automated tests; live-query latency p95 $\approx$ 9.6 ms (budget 2 s).
- Clean GitHub repo + deployment artifacts (Docker / Streamlit Cloud / Render) ready.

Working prototype — the dashboard

Kigali Flood-Pressure Inspection Dashboard — core panels (data-driven preview)



Live metrics (this date)

729

Cells

100

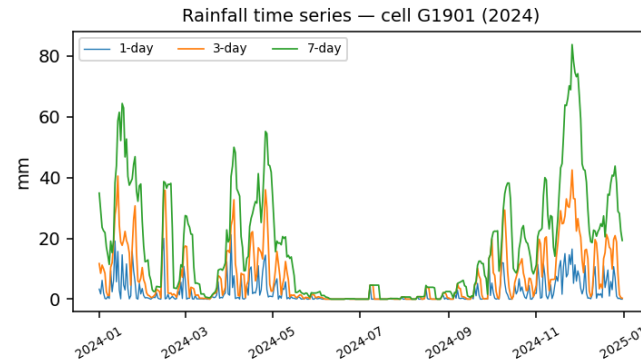
Predicted High

11.8 mm

Mean 3-day rain

0.813

Model macro-F1 (2024)



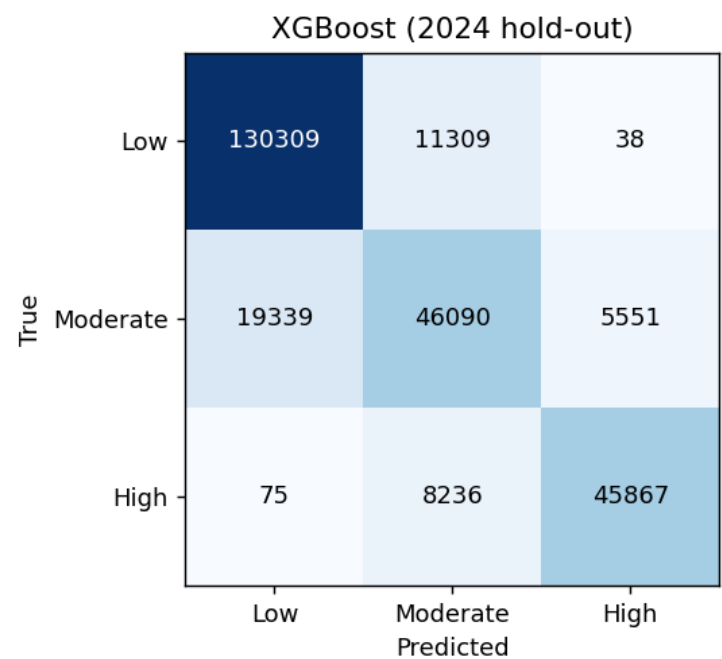
Pick a date → predicted flood-pressure map, metrics, per-cell inspector

- Streamlit app

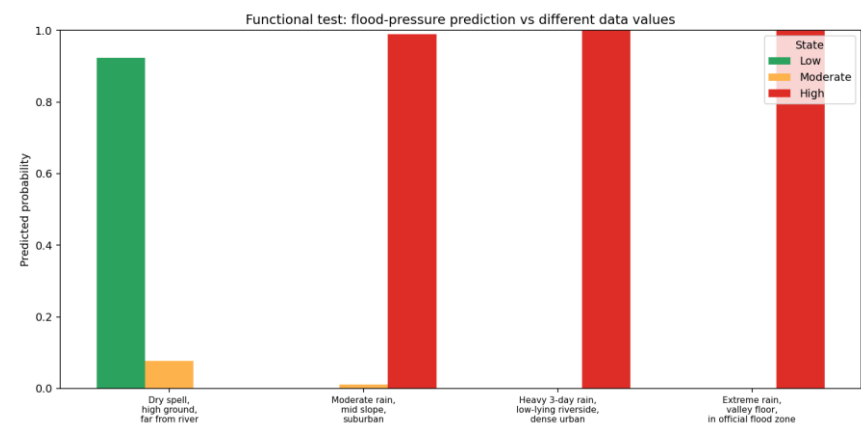
- Any date → Low/Mod/High map over 729 cells
- Toggle official flood polygons
- Inspect a cell: features + class probabilities
- Per-cell Markov transition matrix
- Rainfall series, train/val/test & HMM panels

- Boots headless, serves HTTP 200
- p95 ≈ 9.6 ms / query

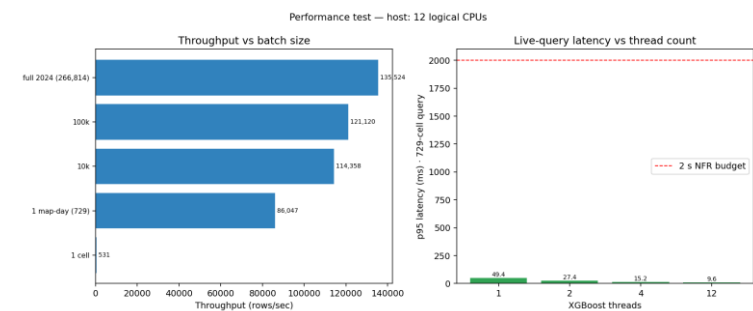
Working prototype — it works on real & varied data



XGBoost confusion (2024 hold-out)



Prediction responds to different data values



Latency well inside the 2 s budget

Summary & next steps

- Thesis demonstrated: open climate + geospatial data + ensemble ML give a time-aware flood-pressure view that beats static maps and rule baselines.
- End-to-end and reproducible: data → models → validation → live dashboard.

Next steps

- Ground-truth against real flood-incident records (biggest validity gain).
- Calibrate probabilities → a meaningful Moderate band; feed forecast rainfall for a 1–7 day outlook.
- Operationalise: scheduled refresh, drift monitoring, alert push; finalise hosting.